

COMP9517: Computer Vision

2026 Term 2

Group Project Specification

Maximum Marks Achievable: 30

The group project is **worth 30% of the total course mark**.

Project work is in Weeks 6-10 with deliverables due in Week 10.

Deadline for submission is Friday 7 August 2026 22:00:00 AET.

Instructions for online submission will be posted closer to the deadline.

Refer to the separate marking criteria for more detailed information on marking.

Introduction

The goal of the group project is to work together with peers in a team of 5 students to solve a computer vision problem and present solutions and results in both oral and written form.

Group members can meet with their tutors once per week in Weeks 6-10 during the usual consultation session hour to discuss progress and get feedback.

Description

Accurate, automated recognition of species from photographs is increasingly important for biodiversity monitoring, ecological research, and citizen science. Online platforms now collect millions of images of plants, animals, and fungi contributed by the public, and there is a strong need for computer vision methods that can identify the species in these images quickly and reliably. One of the central difficulties is fine-grained recognition: many species look similar and differ only in subtle visual cues, while images of the same species can vary widely in pose, lighting, background, and scale. Computer vision methods can help to recognise species at a scale and speed that is impossible to achieve manually.

Task Statement

The goal of this group project is to develop and compare different computer vision methods for **classifying images of living organisms into a fixed set of species**, using a subset of the iNaturalist-2021 dataset (see the Public Dataset section below for the required number of classes and images).

Public Dataset

This project is based on [iNaturalist-2021 \(iNat2021\)](#), a large-scale, fine-grained visual classification benchmark of real citizen-science photographs of plants, animals, and fungi (see reference and link at the end of this document). The full dataset is large and deliberately challenging:

- **10,000 species** (classes), spanning plants, animals, and fungi;
- approximately **2.7 million training images** in the full training set, plus a smaller “mini” training set of about **500,000 images** (50 per species);
- a labelled **validation set of 100,000 images** (10 per species);
- images that vary widely in pose, lighting, background, and scale, with many visually very similar species, which makes the recognition problem genuinely hard.

The full dataset is far too large to train on with the compute available for this project, so you will work with a subset drawn from the iNaturalist-2021 “Mini” set (the official train_mini split), which contains 500,000 images — exactly 50 per species across all 10,000 species. If you want to work with more images and classes, you are free to use the full datasets but need to clarify the experimental details.

Please refer to the original GitHub repository for the official download URLs for both the mini and full versions of the dataset, and more detailed dataset information description. The following links for the mini dataset are provided for reference:

- Training Images [42GB] (https://ml-inat-competition-datasets.s3.amazonaws.com/2021/train_mini.tar.gz): This archive contains 50 example images for each species, resulting in a total of 500,000 images. For each species, it is recommended to use 40 images for training and the remaining 10 images for validation.
- Training Annotations [42MB] (https://ml-inat-competition-datasets.s3.amazonaws.com/2021/train_mini.json.tar.gz): This file provides the annotation information for 500,000 images in train_mini.tar.gz.
- Validation Images [8.4GB] (<https://ml-inat-competition-datasets.s3.amazonaws.com/2021/val.tar.gz>): This archive contains 10 example images for each species, resulting in a total of 100,000 images. These images will be used as the test set for the project.
- Validation Annotations [9.4MB] (<https://ml-inat-competition-datasets.s3.amazonaws.com/2021/val.json.tar.gz>): This file provide the annotation information for 100,000 images in val.tar.gz.

In your project, **your graded experiments must use, at a minimum:**

- at least **500 species (classes, randomly sample 500 species)**;
- at least **50 training images per class (you can split as 40 training and 10 validation)**;
- at least **10 test images per class (based on the validation set of iNat2021)**.

You may use more classes or images than the minimum if your compute allows, but you must **clearly report exactly which classes you used and how many images per class you used** for training, validation, and testing, so that your results are reproducible. Because the labels of the official iNaturalist-2021 test split were never publicly released, use the official validation images as your held-out test data. Keep the training, validation, and test data strictly separate and never cause data leakage.

Method Development

Many traditional (handcrafted-feature) and deep learning-based computer vision methods could be used for this task. You are challenged to use concepts taught in the course and other techniques from literature to develop your own methods and test their performance.

At least **two genuinely different complete methods** must be developed and tested.

Typically, one method is a **traditional pipeline based on handcrafted features and a classical classifier** (for example, Bag-of-Visual-Words with SIFT features and an SVM classifier) and the other is a **deep learning** pipeline (for example, a convolutional neural network). For the deep learning pipeline, you must include **at minimum one** deep learning method, including the model trained from scratch (starting from random initialisation) and that uses pre-trained weights (e.g., ImageNet-pretrained weights).

We do not expect you to develop everything from scratch, but we do encourage you to try different methods, or modifications of existing ones, or use more state-of-the-art methods that have not been presented in the lectures. As a general guideline, the more you implement yourself rather than copy from elsewhere, the better. Note that the originality and depth of your own work can usually be evident from your submitted materials — code, report, and presentation. In any case, always cite your sources, including all papers, tools, and repositories you have used.

Because each class has only a limited number of training images and many species look alike, naïve approaches will quickly plateau. Strong results come from thoughtful choices: feature design, network architecture, data augmentation, transfer learning, and fine-grained recognition techniques.

For all the developed methods, try to make sure the models are sufficiently trained and converged.

Comprehensive Method Development

If you are aiming for higher marks (23–26), we expect to see sufficient **variety** in the methods you develop. *For example, as a group*, you could develop the following:

- **Traditional (handcrafted-feature) methods**, comparing **different** handcrafted feature descriptors (e.g., colour histograms/moments, LBP, HOG, Bag-of-Visual-Words with SIFT) and classical classifiers (e.g., SVM, Random Forest).
- **Deep learning-based methods**, comparing **different** deep neural network **architectures** and data **augmentation** methods, and comparing transfer learning (training with pre-trained weights) against training from scratch.

You should ensure that the training converges properly and that your model achieves reasonable performance; you are not expected to achieve state-of-the-art results. Your report should clearly document the training process and demonstrate that the model was trained properly.

We expect rigorous **ablation studies** (systematically varying one design choice at a time), detailed **reporting of the training process/dynamics** (for example, loss and accuracy curves

on the training and validation sets), and a careful **error analysis** (for example, a confusion matrix and an inspection of the hardest, most-confused species pairs).

Advanced Method Development

If you are aiming for even higher marks (26 and above), we expect to see some original ideas and research skills. **This term we are particularly interested in the following four directions**, and a strong project will investigate one or more of them in depth. Investigating one of these directions in depth can earn 26–27; investigating more than one can earn 28 and above.

1. **Explainability with Grad-CAM (deep learning method only).** Use [Grad-CAM](#) (and/or related class-activation-mapping techniques) to visualise and analyse which regions of an image drive your deep model's predictions. Investigate whether the network attends to the organism itself rather than to the background or surrounding context. Compare the activation maps for correctly versus incorrectly classified images, and for confusable species pairs (for example, two species from the same genus). Use these visualisations to explain failure cases and to support concrete claims about what your model has actually learned, rather than presenting attention maps without interpretation.
2. **Robustness to image degradation at test time.** Study how your trained models behave when the **test images** are degraded in ways that occur in practice — for example, additive (Gaussian) **noise**, Gaussian or motion **blur**, and optionally reduced brightness/contrast, or JPEG compression (at least 4 different types of degradations with various parameters, such as noise/blur strengths). Apply these degradations **only to the test images** — do not modify the training data and do not retrain the model. Sweep across several severity levels for each degradation type, and report how top-1 accuracy and macro-averaged F1 fall as a function of degradation type and severity (for example, as performance-versus-severity curves). Discuss which of your methods is the most robust and why, and relate this to how each method represents the image. *(As an additional comparison you may also examine whether training-time augmentation improves robustness — but the core study must keep the model fixed and degrade only the test set.)*
3. **Effect of the number of classes.** Investigate how performance changes as the recognition problem is scaled up to more species. Starting from your base subset, construct additional controlled subsets containing **more classes** (for example, doubling and then quintupling the number of classes). To keep the data and compute budget manageable, you may **reduce the number of images per class** as the number of classes grows (for example, keeping the total number of training images roughly constant). Analyse how top-1 accuracy and macro-F1 change as the task becomes more fine-grained and as per-class data shrinks, and discuss the trade-off between class granularity and per-class sample size. For this experiment you may draw additional classes and/or images from iNaturalist-2021 (for example, the official train_mini and val splits, or iNaturalist Open Data). **Clearly document exactly how you built each subset** — the class lists, the number of images per class, and the random seed — so that your experiments are fully reproducible. This additional data is for the scaling study only; your primary results must still be reported on your base subset's test set.
4. **Continual (class incremental) learning (deep learning method only).** Split the species in your subset into several sequential groups (such as 10 or 20 groups with 10 classes in

each one) and train your deep model on them one group at a time. (1) First, train continually on each new group in turn and **verify catastrophic forgetting** — measure how much the **accuracy** on the earlier species drops as new species are added, using a standard *average forgetting measure* (De Lange et al.), and report performance on *old* versus *new* classes at each stage. (2) Then, apply a simple **replay-based method** — keep a small memory of earlier images and mix them into the training of later groups — and show how much it reduces forgetting (Chaudhry et al.). You may use accuracy-based metrics for this topic. Discuss the trade-off between the size of the replay memory and the amount of forgetting. This study runs entirely within your existing subset, so no additional data is required. For this topic, you can restrict your experiments to a relatively small set of classes, such as 100 or 200 in total, to keep the experiments manageable.

These four are not the only options. Other valuable original directions include:

- **Fine-grained-specific methods**, such as bilinear / second-order pooling, or attention- and part-based models.
- A **long-tailed / class-imbalanced variant**: resample the subset into a realistic long-tail and study reweighting, resampling, and augmentation.
- **Metric learning / few-shot learning** (natural here, since there are only a few images per class): triplet or contrastive embeddings, with a nearest-class-mean classifier in the embedding space.
- **Weakly-supervised segmentation**: using only the image-level class labels (no pixel masks), segment the organism from the background — for example by thresholding class-activation maps (CAM/Grad-CAM) into an object mask — and assess qualitatively or quantitatively how well the segmented region matches the organism.

Do discuss with your group and see if you can come up with other original ideas that add value to your methods and results. You can further discuss your ideas with the tutors during the weekly consultation hours.

Training and Testing

Keep your data split into training, validation, and test subsets, and never mix them up or cause data leakage. Whether your methods require training (supervised methods) or not (unsupervised methods), they will probably have (hyper)parameters that you need to fine-tune to get optimal performance. Use the training data (holding out part of it as a validation set) for this, and use the test set only for final performance evaluation. Using (partly) the same data for both training/fine-tuning and testing leads to biased results that are not representative of real-world performance.

To assess the performance of your methods, compare the predicted classes quantitatively with the ground-truth classes using appropriate metrics. Specifically, **compute at least the top-1 and top-5 accuracy, the overall accuracy (and, optionally, the mean per-class / balanced accuracy), and the macro-averaged precision, recall, and F1 score**

(https://www.cl.uni-heidelberg.de/~opitz/pdf/metric_overview.pdf; https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html).

Because the task is fine-grained with many similar classes, place particular emphasis on

macro-averaged F1 and a confusion-matrix analysis (e.g., a visualization of for all classes and more detailed analyses for selected subset of classes), which reveal weak per-species performance that overall accuracy alone can hide. Furthermore, compare the training and testing time (vs performance) of the different methods.

Results and Discussion

Show the quantitative results in your video presentation and written report (see deliverables below). Also show representative examples of successful results as well as examples where your methods failed (no method generally yields perfect results). Give some explanation why you believe your methods failed in these cases. Also, if one of your methods clearly works better than the other(s), discuss possible reasons why. Finally, discuss some potential directions for future research to further improve performance.

Practicalities

A suitably small subset (on the order of a few hundred megabytes) makes method training and testing comfortably feasible on the free version of Google Colab (typically a T4 GPU). A practical workflow is to store your data in Google Drive (or download it), copy it to the local Colab disk at the start of each session, and then train. A transfer-learned CNN converges faster than training from scratch. Writing code to print out and/or visualize the training loss or anything you want to monitor during training. The classical pipeline run on the CPU. You can decide to use Colab or your local CPU. We have tested the project setup on Google Colab and confirmed that it is feasible there.

Training a proper size of CNN on the data takes a while to converge. So monitor the training status and make sure the models are trained sufficiently (not significantly under fitting). As training may take some time, so plan your experiments well in some ways — for example (optional), start with a small subset of the data, save intermediate checkpoints regularly (including the model weights and optimiser state), and learn how to resume training from a checkpoint. These steps are not required, but planning for them will make your workflow smoother and more robust to interruptions.

You may also run on your modern desktop or laptop computer with GPUs better than Colab.

You may not, however, use any data from outside iNaturalist-2021 in your graded evaluation, except in a self-designed advanced research topic.

Deliverables

The deliverables of the group project are 1) a video presentation, 2) a written report, and 3) the code. The deliverables are to be submitted by only one member of the group, on behalf of the whole group (we do not accept submissions from multiple group members).

Video

Each group must prepare a video presentation of at most 10 minutes (any content beyond

that will be ignored). The presentation must start with an introduction of the problem and then explain the used methods, show the obtained results, and discuss these results as well as ideas for future improvements. For this part of the presentation, use PowerPoint slides to support the narrative.

Following this part, **the presentation must include a demonstration of the methods/software in action**. Of course, some methods may take a long time to compute, so you may record a live demo and then edit it to stay within time.

All group members must present (points will be deducted if this is not the case). It is up to you to decide who presents what (introduction, methods, results, discussion, demonstration). In order for us to verify that all group members are indeed presenting, each student presenting their part must be visible in a corner of the presentation (live recording, not a static head shot), and when they start presenting, they must mention their name.

Overlaying a webcam recording can be done using either the video recording functionality of PowerPoint itself (see for example [this YouTube tutorial](#)) or using other recording software such as [OBS Studio](#), [DaVinci Resolve](#), [Adobe Premiere](#), and many others, or the recording functionality of Zoom or MS Teams. It is up to you (depending on your preference and experience) which software to use, as long as the final video satisfies the requirements mentioned above and below.

The entire presentation must be in the form of a video (720p or 1080p MP4 format). Note that video files can become quite large (depending on the level of compression used). **To avoid storage problems for this course, the video upload limit is 100 MB per group, which should be more than enough for this type of presentation.** If your video file is initially larger, use tools like [HandBrake](#) to re-encode with higher compression. For example, in Handbrake, choose the "Very Fast 720p30" Preset and then go to the Video tab to lower the Quality as needed. That way, even a 1-hour video presentation can be compressed to under 100 MB without significant loss of quality. Minor compression artifacts will not affect your mark.

The video presentations will be marked offline (there will be no live presentations). If the markers have any questions, they may contact the group members by email for clarification.

Report

Each group must also submit a written report in [CVPR format](#) (it is a LaTeX-based template), **rendered and submitted in PDF** (use the official CVPR template, use `"\usepackage[pagenumbers]{cvpr}"` – check within the LaTeX template, main.tex). Please modify the contents to fit the requirement and contents of your report. **We only accept PDF submissions rendered using this LaTeX template. Reports that do not follow this format may receive a mark of zero.** The main text of PDF may be at most 10 pages, including all diagrams, figures, and tables, but excluding references; you may add any number of additional pages *for references only*.

The report must be submitted as a PDF file and include:

- **Introduction:** Present a brief introduction including the relevance and your understanding of the task and the dataset.

- **Literature Review:** Review relevant techniques in literature, along with any necessary background to understand the methods you selected.
- **Methods:** Motivate and explain the selection of the methods you implemented, using relevant references and theories where necessary.
- **Experimental Results:** Explain the experimental setup you used to test the performance of the developed methods and the results you obtained. Report the main hyperparameters for models and optimizers you used in experiments.
- **Discussion:** Provide a discussion of the experimental results and findings, comparing the different methods you developed.
- **Conclusion:** Summarise what worked and what did not work, any limitations of the developed methods, and recommend future work.
- **References:** List the literature references and other resources used in your work. The references section does not count toward the 10-page limit.

The upload limit for the report is 10 MB per group. If your report is initially larger than that, you may need to resize images or increase the compression.

Code

The source code of the developed software must be submitted as a ZIP file. The file should mainly contain your own code. Libraries or code obtained from other sources should be clearly described in a README file. Include proper documentation about how to run the code, use inline comments, and make the code well structured.

The upload limit for the source code is 40 MB per group. To stay within this limit, do not include trained models, input images, or result images.

Student Contributions

As a group, you are free in how you divide the work among the group members, but all group members must contribute roughly equally to the method development, coding, making the video, and writing the report. For example, it is unacceptable if some group members only prepare the video or report without also contributing to the methods and code.

An online survey will be held in Week 11 (after the project deliverables submission deadline) allowing students to anonymously evaluate the relative contributions of their group members to the project. The results will be reported only to the LIC and the Course Administrators, who at their discretion may moderate the final project mark for individual students if there is sufficient evidence that they contributed substantially less than the other group members.

Group Requirements

Be sure to carefully check which group you have been assigned to by the course admins as announced on WebCMS. If you did not sign up for a group before the cut-off date, you have been randomly assigned to a group by the course admins. You are required to work with your assigned group. Failing to do so will result in a 50% penalty on your project mark.

If you did form a group with other students ahead of time, but you or some other members

forgot to register as a member of the group on WebCMS before the cut-off date, meaning you failed to let the course admins know, you have now been randomly assigned to some other group and are required to work with that group. Failing to do so and continuing working with the group you had in mind will result in a 50% penalty on your project mark.

If you registered a group on WebCMS but your group had less than 5 members, the course admins may have randomly assigned additional students to your group. You are required to work with all students in your group. Excluding certain members because they were not part of the group you had in mind will result in a 50% penalty on your project mark.

Dataset

The dataset to be used in the group project is a subset of the following dataset:

Grant Van Horn, Elijah Cole, Sara Beery, Kimberly Wilber, Serge Belongie, Oisin Mac Aodha. Benchmarking Representation Learning for Natural World Image Collections. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2021.

https://github.com/visipedia/inat_comp/tree/master/2021

Check the “Data” section on the GitHub repo page to find the URL and command for accessing the full or mini dataset.

References

The following papers may be helpful in getting a deeper understanding of the problem and exploring potential methods:

Grant Van Horn et al. *The iNaturalist Species Classification and Detection Dataset*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.
<https://doi.org/10.1109/CVPR.2018.00914>

Ramprasaath R. Selvaraju et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. IEEE International Conference on Computer Vision (ICCV), October 2017. <https://doi.org/10.1109/ICCV.2017.74>

Dan Hendrycks and Thomas Dietterich. *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. International Conference on Learning Representations (ICLR), May 2019. <https://arxiv.org/abs/1903.12261>

Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc’Aurelio Ranzato. *On Tiny Episodic Memories in Continual Learning*. arXiv:1902.10486, 2019. <https://arxiv.org/abs/1902.10486>

Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory G. Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(7):3366–3385, 2022. <https://doi.org/10.1109/TPAMI.2021.3057446>

Qingsen Yan, Dong Gong, Yuhang Liu, Anton van den Hengel, and Javen Qinfeng Shi. *Learning Bayesian Sparse Networks with Full Experience Replay for Continual Learning*.

IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2022.
<https://arxiv.org/abs/2202.10203>

Huiyi Wang, Haodong Lu, Lina Yao, and Dong Gong. *Self-Expansion of Pre-Trained Models with Mixture of Adapters for Continual Learning*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2025. <https://arxiv.org/abs/2403.18886>

Copyright: UNSW CSE COMP9517 Team. Reproducing, publishing, posting, distributing, or translating this assignment is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.

Released: 1 July 2026